Aula 2 – Analise da Complexidade

Recursividade

Métodos de Análise de Algoritmos

A Notação O

Exercícios

Recursividade

- Em programação todos os algoritmos que não são recursivos chamam-se algoritmos iterativos
- Recursividade consiste em uma função chamar-se a ela própria.
- Erros comuns com a recursividade: **esquecer-se** da condição de paragem.

Recursividade

Recursividade como a 4ª forma de fazer ciclos

Exemplo: mostrar os numero de um a 10

```
void mostra() {
        int i = 1;
        while (i <= 10) {
                System.out.println(i);
                i++;
        }
}
```

```
void mostra() {
        int i = 1;
        do {
                System.out.println(i);
                i++;
        } while (i <=10);
}
```

```
void mostra() {
        for(int i = 1; i <= 10; i++ {
                System.out.println(i);
        }
}
```

```
void mostra() {
        mostraAux(1);
}
void mostraAux(int  i) {
                System.out.println(i);
                i++;
                if (i <= 10)
                        mostraAux(i);
}
```

Recursividade

Calcular o factorial de um numero:

Algoritmo Iterativo:

```
int factorial(int num) {
        int f = 1;
        for (int i = 1; i <= num; i++)
                f = f * i;
        return f;
}
```

Algoritmo Recursivo:

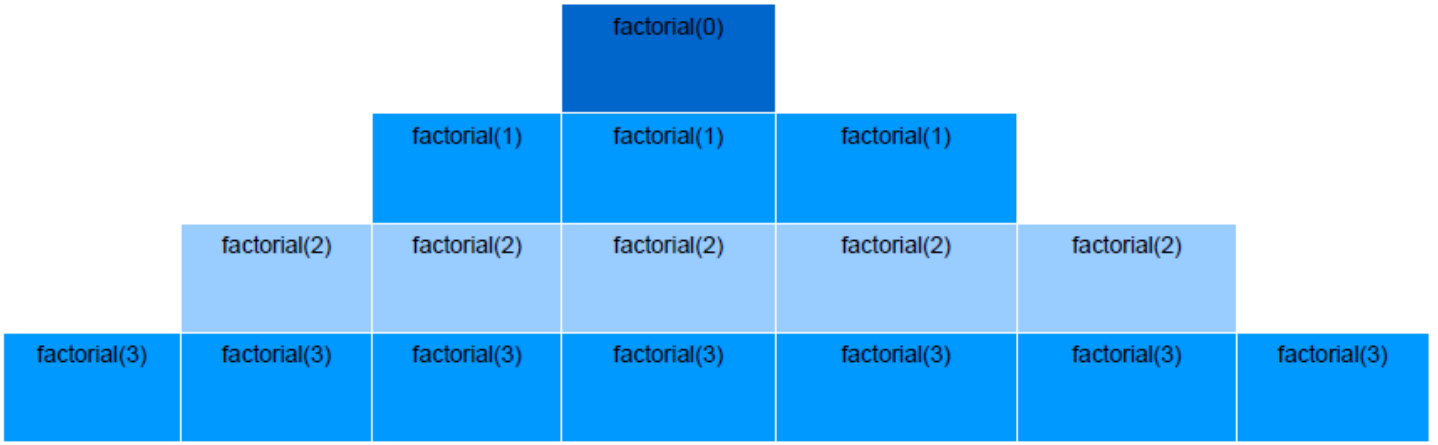
```
int factorial(int num) {
        if (num == 0) return 1; // condição de paragem
        else return num*factorial(num-1);
}
```

Recursividade

Como funciona a recursividade:

```
int factorial(int num) {
    if (num == 0) return 1; // condição de paragem
    else return num*factorial(num-1);
}
```

```
factorial(3);
    → return 3*factorial(2);
        → return 2*factorial(1);
            → return 1*factorial(0)
                → return 1;
            ← return 1*1;
        ← return 2*1;
    ← return 3*2;
6
```



os algoritmos recursivos são algoritmos que consomem muita memória porque temos que guardar todas as chamadas da função

Algoritmos Recursivos

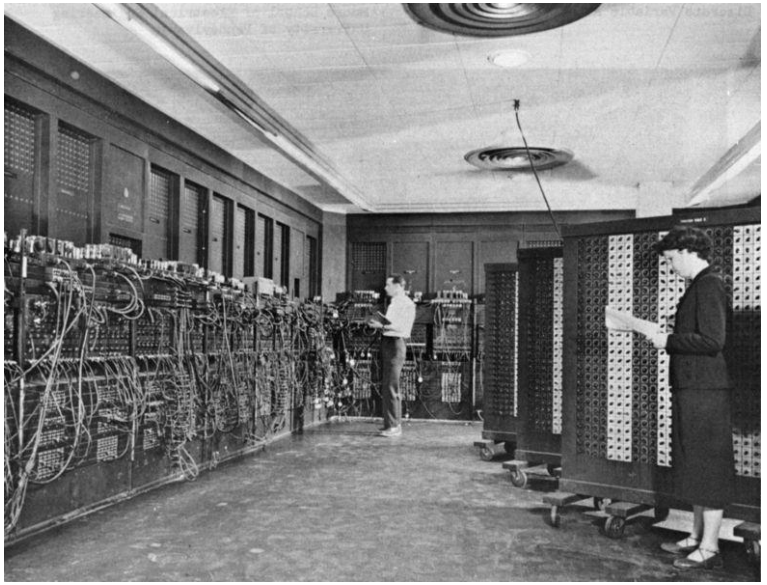
Desvantagens:

- Consomem mais memoria que os iterativos

Vantagens:

- Mais intuitivos, logo mais fáceis de os atingir para resolver problemas complexos

Porque se estuda a Complexidade dos Algoritmos





Imagine um programa em Java para os antigos Eniac ou Univac mostrarem os números de 1 a 10?

Porque se estuda a Complexidade dos Algoritmos

Algoritmo A:

```
for(int i = 1; i <= 10; i++)
System.out.println(i);
```

Algoritmo B:

```
System.out.println(”1”);
System.out.println(”2”);
System.out.println(”3”);
System.out.println(”4”);
…
System.out.println(”10”);
```

Algoritmo C:

```
System.out.println(”1\n2\n3\n4\n5\n6\n7\n8\n9\n10”);
```

Qual é o Melhor A, B ou C?

Porque se estuda a Complexidade dos Algoritmos

Algoritmo A:

Uma atribuição (i = 1)

Onze comparações (i <= 10)

Dez somas (i+1)

Dez atribuições (i = i + 1)

Dez instruções para mostrar no ecrã

Algoritmo B:

Dez instruções para mostrar no ecrã

Algoritmo C:

Uma instrução para mostrar no ecrã

Qual é o Melhor A, B ou C?

Porque se estuda a Complexidade dos Algoritmos

Algoritmo A:

Qual é o Melhor A, B ou C?

Usa uma variável int, gasta 4bytes de memoria

Algoritmo B:

Não usa nenhuma variável

Algoritmo C:

Não usa nenhuma variável

Porque se estuda a Complexidade dos Algoritmos

O Melhor é o Algoritmo C.

Os primeiros computadores tinham pouca capacidade de processamento e pouca memoria. Todas as instruções e todas as variáveis eram criteriosamente escolhidas.

Estuda-se a complexidade dos Algoritmos para:

- Escolher entre vários algoritmos o mais eficiente para implementar
- Desenvolver novos algoritmos para problemas que já tem solução
- Melhorar os algoritmos devido ao aumento constante dos problemas a serem resolvido

Métodos de Analise de Algoritmos

- Métodos a Priori: Métodos que podemos usar antes de implementar o algoritmo e executá-lo.

- Métodos a Posteriori: Métodos que usamos depois de executar o algoritmo.

Métodos a Posteriori

- Cronometrar o tempo de execução do algoritmo
- Verificar o espaço em memória que o algoritmo usou.

- Depende de máquina para máquina.

Métodos a Posteriori

```
public class Cronometer {
private long startTime, elapsedTime;
private boolean run;
public Cronometer() {
        elapsedTime = 0;
        run = false;
}
public  void start() {
                startTime = System.currentTimeMillis();
                elapsedTime=0;
                run = true;
}
public  void stop() {
        if (run == true) {
                elapsedTime += System.currentTimeMillis() - startTime;
                run = ! run;
        }
}
public long getElapsedTime() {
        return elapsedTime;
}

}
```

Métodos a Posteriori

```
public static void main(String args[]) {
  Chronometer cr = new Chronometer();
  cr.Start();
  // Colocar aqui o código a anlisar
  cr.Stop();
  System.out.println( " Tempo:" + cr.getElapsedTime() );
}
```

Métodos a Posteriori

```
public static void main(String args[]) {
        List<Person> list = new ArrayList<Person>();
        for (int i = 0; i <= 100000; i++) {
           list.add(new Person("Ana", "Maria"));
        }
        // Get the Java runtime
        Runtime runtime = Runtime.getRuntime();
         // Run the garbage collector
        runtime.gc();
        // Calculate the used memory
        long memory = runtime.totalMemory() - runtime.freeMemory();
        System.out.println("Used memory is bytes: " + memory);
}
```

Métodos a Priori

- Contar o número de instruções (Complexidade Temporal)
  1. 1 atribuição
  2. 1 atribuição + (N+1) comparações + N somas + N atribuições
  3. N multiplicações + N atribuições
  4. 1 atribuição

2N + 3 atribuições

N Somas

N Multiplicações

N + 1 Comparações

Total: 5N + 4 instruções

```
int power1 (int x, int n){
1.    int result=1;
2.    for(int i=0; i< n ; i++)
3.       result *=x;
4.     return result;
}
```

Métodos a Priori

- Contar o numero de variáveis utilizadas (Complexidade Espacial)
- int x
- int n
- int result
- int i
- int (return)
- Total: 5 inteiros * 4 bytes = 20 bytes

```
int power1 (int x, int n){
1.   int result=1;
2.   for(int i=0; i< n ; i++)
3.      result *=x;
4.    return result;
}
```

A notação O

Aplica-se na complexidade temporal

Da melhor para a pior:

- O(1) : Constante
- O(logn) : Logarítmica
- O(nlogn) : nlogn
- O(n) : Linear
- O(n^k): Polinomial
  - O(n^2) : Quadrática
  - O(n^3) : Cúbica
- O(k^n) : Exponencial

- Usa-se a notação O para analisar o pior caso, a notação para o Θ caso médio e a notação Ω para o melhor caso.

O(1) - Constante

- Complexidade Constante – Algoritmos Óptimos
- Demoram sempre o mesmo tempo independentemente do tamanho do problema

- Exemplo: Calcular a soma de todos os números naturais entre um intervalo

```
int Soma( int min , int max) {
        return  (max – min +1 )/2 * (max + min);
}
```

O(logn) - Logarítmica

- Complexidade Logarítmica – Algoritmos muito bons.
- Resultam da divisão de problema em sub-problemas menores – Dividir para conquistar
- Normalmente são algoritmos recursivos
- Exemplo: Considere o seguinte jogo:
- Pense num número entre 1 e 16
- Por cada número que eu disser dizes-me se o número que pensaste é maior ou menor

```
bool Adivinha( int min , int max, int segredo){
    int palpite = (min + max) /2;
    if( palpite == segredo)
        return true
    if( segredo > palpite)
        return Adivinha(palpite+1, max, segredo)
    else
        return Adivinha(min, palpite-1, segredo)
}
```

O(n) - Linear

- Complexidade Linear – Algoritmos bons.
- São o resultado do processamento individual de cada elemento da entrada

- Exemplo: Calcular a soma dos elementos de um array

```
int Somar( int V[ ] , int N){
int soma=0;
   for(int i=1 ; i < N ; i++)
       soma +=V[i]
   return soma;
}
```

O(nlogn) - nlogn

- Complexidade nlogn
- São algoritmos que à semelhança do logn, dividem o problema em sub-problemas menores, mas que juntam a solução de cada sub-problema para obterem a solução final.

- Exemplo: Algoritmos de ordenação como o mergesort e quicksort

O(n^k) – Polinomial

- Complexidade Polinomial
  - Se k = 2 , O(n^2) – Quadrática
  - Se k = 3, O(n^3) - Cúbica
- Resultam de ciclos encadeados
- Pouca utilidade se K >= 3

- Exemplo: Procurar um valor dentro de uma matriz, O(n^2)

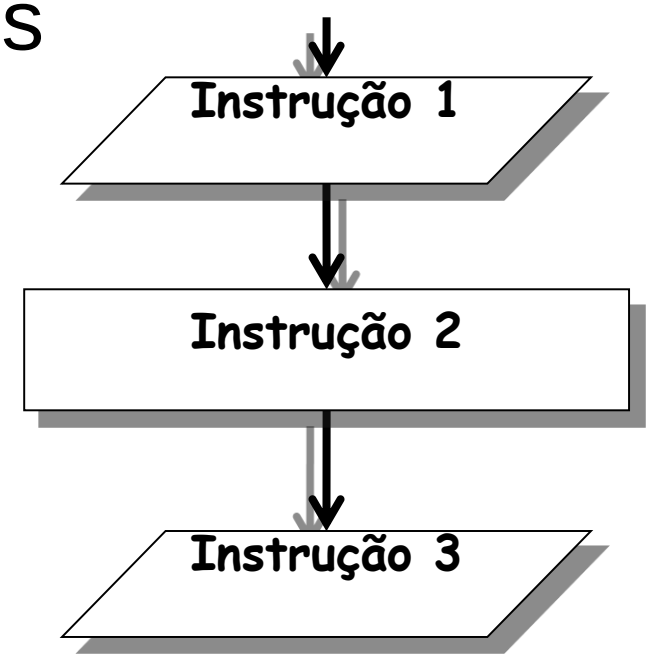
O(k^n) – Exponencial

- Complexidade Exponencial
- Resultam de Algoritmos de força bruta
- Para a resolução de cada problema são criados mais k problemas.
- Exemplo: Fibonacci recursivo, saber o n termo da serie de Fibonacci

```
int Fibonacci(int n)
{
  if(n==1 || n==2)
     return 1;
  else
     return Fibonacci(n-1) + Fibonacci(n-2);
}
```

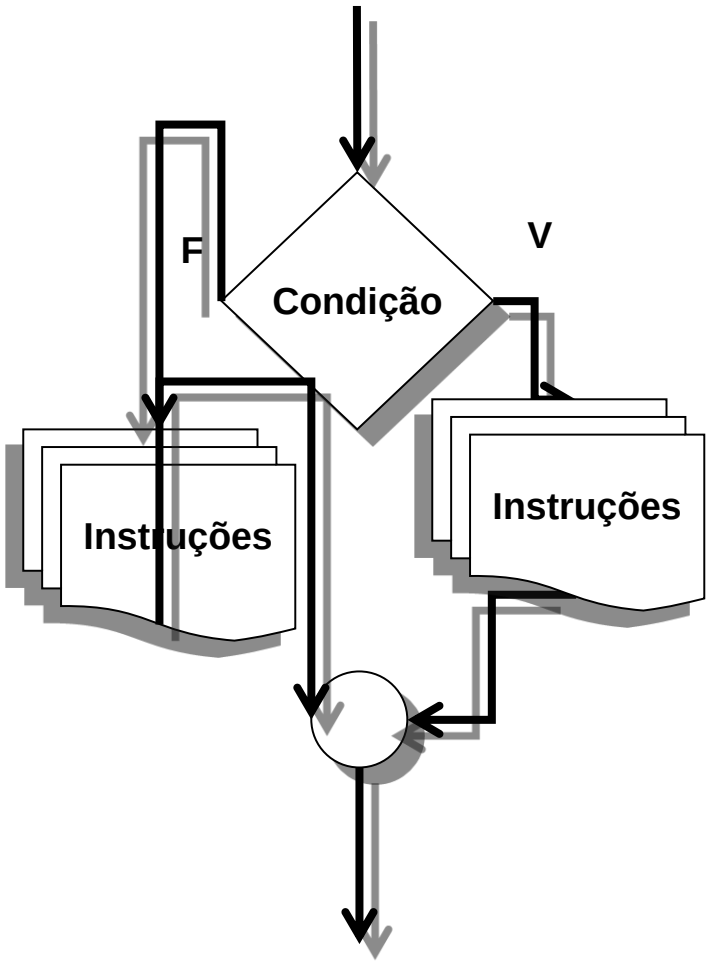
Regras gerais para o Cálculo da Complexidade

- Instruções consecutivas
- Calcular o máximo
  - Exemplo **O(fn1) + O(fn2) = O(max(fn1,fn2))**
- Chamada de funções Funções
- Complexidade da função



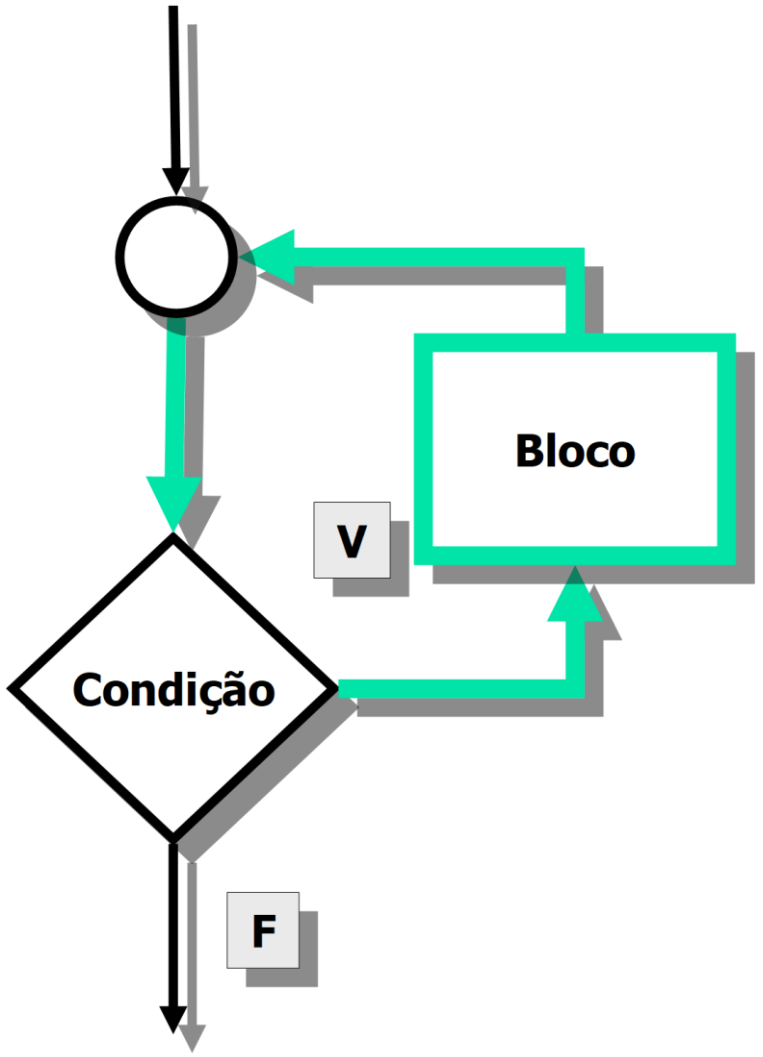
Regras gerais para o Cálculo da Complexidade

- Desvios
- Considerar o ramo com
- maior complexidade



Regras gerais para o Cálculo da Complexidade

- Ciclos
  - complexidade do bloco multiplicado pelo número de iterações.
- Ciclos encadeados
  - Analisar de dentro para fora.
- Multiplicação da complexidade de cada um dos blocos



Regras gerais para o Cálculo da Complexidade

- Recursividade
- Análise de cada ramo da recursão

- Casos
- A recursividade reduz o problema numa quantidade constante
  - Linear O(N)
- A recursividade reduz o problema numa fracção N
  - Logaritmo O(logn)
- A recursividade gera N novos problemas
  - Exponencial – o(k^n)

A notação S

Aplica-se na complexidade espacial

Da melhor para a pior:

- S(1) : Constante
- S(n) : Linear
- S(n^k): Polinomial
  - S(n^2) : Quadrática
  - S(n^3) : Cúbica

Exercícios

Qual é a complexidade espacial (notação S) e a complexidade temporal (notação O)

Exercícios

```
Funcao Factorial(  num)
    total <- 1
    n <- 1
    enquanto n <= num faz
        total <- total * n
        n<- n + 1
    fimEnquanto
    retorna total
FimFuncao
```

```
Funcao SomaInt(  num)
    se num = 1  então
        retorna 1
    senão
        resto <- SomaInt(num -1)
        retorna num + resto
    Fimse
FimFuncao
```

Exercícios

```
Funcao F2( n)
   total <- 0
   para i de 1 ate n
      para k de i ate n
         total <- total + i
      proximo
   proximo
fimFuncao
```

```
Funcao F3( n)
   total <- 0
   para i de 1 ate n*n
         total <- total + F2(i)
   proximo
fimFuncao
```

Exercícios

```
Hanoi( n,  orig, dest,  aux)
   Se n = 1 então
          escrever "disco 1 " , orig , " -> " , dest
   senão
        Hanoi(n-1, orig, aux, dest);
         escrever "disco" , n , orig , " -> " , dest
    Hanoi( n-1, aux, dest, orig);

   FimSe
FIM
```